

Contents

depthed — Interview Guide (Deep Dive)	1
1. The 60-second pitch (say this out loud)	1
2. System map	2
3. End-to-end lifecycle of one lesson (the money walkthrough)	3
4. Backend deep dive	4
4.1 The graph (server/graph.py)	4
4.2 Checkpointing & the pooler (the subtle production story)	4
4.3 Two persistence layers (don't conflate them)	4
4.4 Auth (server/deps.py)	5
4.5 Streaming (sse-starlette)	5
4.6 Rate limiting (server/ratelimit.py)	5
4.7 Config, logging, tracing	5
5. Frontend deep dive	5
5.1 Structure (client/src/app, App Router)	5
5.2 Auth & data fetching	5
5.3 The streaming hook (lib/hooks/use-lesson.ts)	6
5.4 Server state (TanStack Query)	6
6. Key design decisions & trade-offs	6
7. Data model (server/models.py)	7
8. Reliability hardening pass (great "tell me about a bug" material)	7
9. Known limitations / what I'd do next	7
10. Interview Q&A bank	8
11. Quick file map (where to point during a screen-share)	9

depthed — Interview Guide (Deep Dive)

A code-grounded walkthrough of how depthed is **actually built**, the decisions behind it, the trade-offs, and a Q&A bank to rehearse with. This complements [architecture.md](#) — that doc is the *target design (RFC)*; this doc describes the *implementation as it stands* and flags where the two diverge.

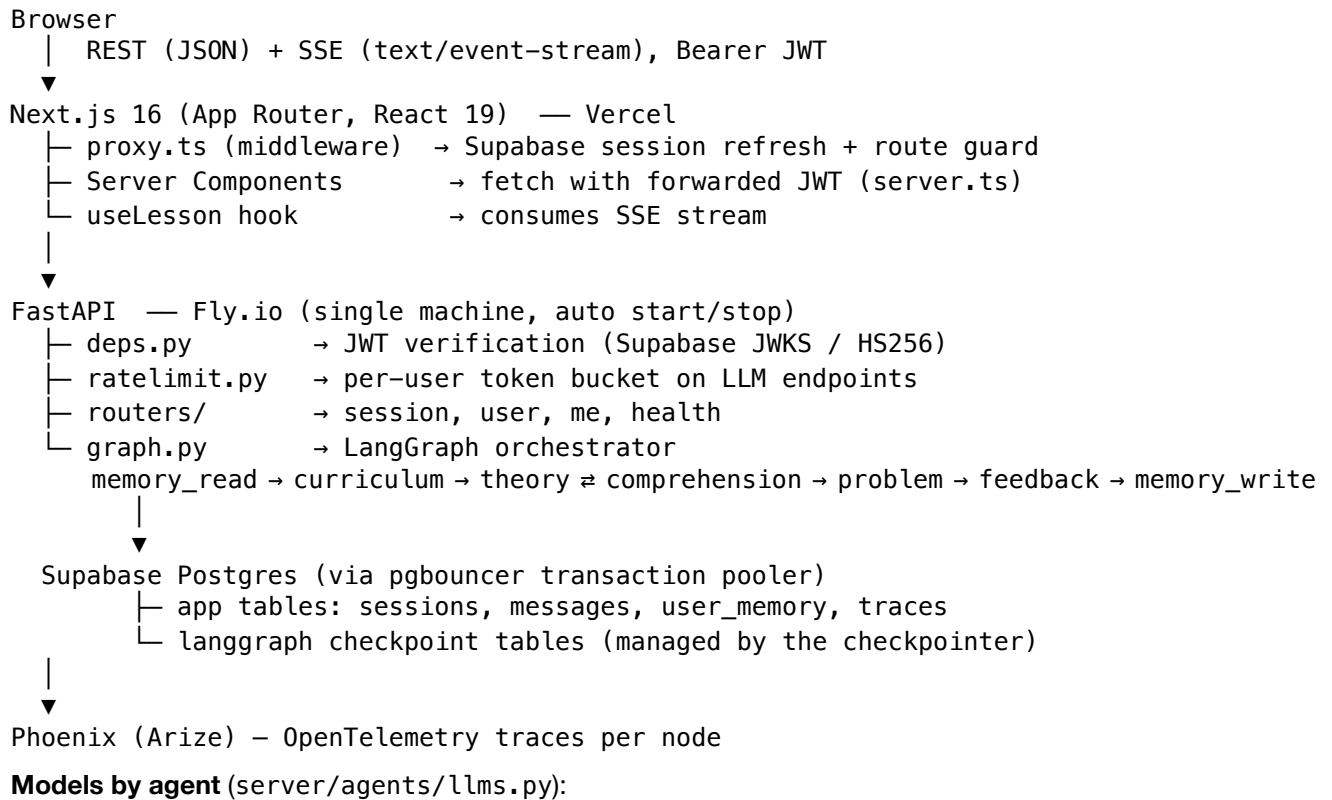
One-liner: depthed is a multi-agent AI tutor for backend devs. Each topic becomes a progressively-revealed lesson page — *Theory* □ *Comprehension Check* □ *Problem* □ *Solution* □ *Feedback* — produced by a LangGraph agent graph, streamed to a Next.js client over SSE, with per-session state persisted in Postgres so the graph can pause mid-lesson and resume across HTTP calls.

1. The 60-second pitch (say this out loud)

“depthed teaches backend topics through a Socratic loop instead of a chat. The backend is a FastAPI service wrapping a **LangGraph** state machine of five agents. The graph **interrupts** at the comprehension check and the problem submission — it literally pauses, persists its state to Postgres via the LangGraph checkpointer, and resumes on the next HTTP request, keyed by `thread_id = session_id`. Agent output streams to the client token-by-token over **SSE**. I route each agent to a different model by job — cheap models for planning/memory, GPT-4o for code generation, Claude Sonnet for explanation and feedback quality. Everything runs on Supabase Postgres — app data, auth, and graph checkpoints in one database — and every node is traced to Phoenix.”

If you only memorize one thing, memorize **interrupt + checkpoint + resume**. That's the technically interesting core and the thing an interviewer will dig into.

2. System map



Agent	Model	Why
Curriculum	gpt-4o-mini	Planning/routing — low reasoning, runs every session, cost-sensitive
Memory (r/w)	gpt-4o-mini	Structured read/write, no creative reasoning
Theory	claude-sonnet-4-6	Explanation quality is the product

Agent	Model	Why
Problem	gpt-4o	Generated code must be correct/runnable Honest, non-sycophantic critique Cheap nudge, not a full solution
Feedback	claude-sonnet-4-6	
Hint endpoint	gpt-4o-mini	

3. End-to-end lifecycle of one lesson (the money walkthrough)

This is the most likely “walk me through what happens when...” question.

1. **Start.** Client `POST /session/start {topic}`. The handler inserts a sessions row (`status="active"`) and returns `session_id`. No graph work yet. (`routers/session.py:start`)
2. **First stream.** Client opens `POST /session/stream {session_id, input: null}`.
 - The handler loads the session **scoped to the authenticated user** (`_load_owned_session`) $\square$ 404 if not theirs (this is the IDOR guard).
 - `_event_stream` calls `graph.aget_state(cfg)`. Empty $\square$ first run, so it streams the graph with `{user_id, topic}` as input.
 - Graph executes `memory_read` $\rightarrow$ `curriculum` $\rightarrow$ `theory_explain`. The theory node **streams tokens** via LangGraph’s custom stream writer; each becomes an SSE `{type:"token", section:"theory"}` event.
 - The graph is compiled with `interrupt_before=["comprehension", "feedback"]`, so it **halts before** the comprehension node. `aget_state().next` is non-empty $\square$ the handler emits `{type:"done", status:"waiting", next:"comprehension"}` and the HTTP response ends. **The graph state is checkpointed in Postgres.**
3. **Comprehension answer.** Client `POST /session/stream {session_id, input:"<answer>"}`.
 - `aget_state` now has values $\square$ the handler calls `update_state(cfg, {pending_input: answer})` and resumes (`stream_input=None`).
 - The comprehension node grades the answer with Sonnet (temp 0) and sets `comprehension_verdict`. A **conditional edge** (`_route_after_check`) routes: passed $\rightarrow$ problem; failed & attempts < 2 $\rightarrow$ theory (re-explain); failed & attempts $\geq$ 2 $\rightarrow$ problem (give up gracefully and move on).
 - If routed to problem, the graph streams the problem then **interrupts before feedback** $\square$ `{type:"done", status:"waiting", next:"feedback"}`.
4. **Solution submission.** Same `/session/stream` call with the solution as input.
 - Resumes into `feedback_review` (streams critique), then `memory_write` updates `user_memory` (topic completed/struggling, suggested next), then END.
 - On reaching END (`snap.next` empty), `_finalize` writes the durable artifact: deletes prior messages for the session, re-inserts agent/user messages from the final state, records the Phoenix `trace_id`, and flips the session to completed.

- Emits `{type:"done", status:"complete"}`.

5. **Re-open later.** GET `/session/{id}` rebuilds `LessonData` from the **checkpoint state** (the source of truth for what the page shows), not the messages table.

Key insight to articulate: the lesson is a *long-lived, multi-request workflow*. HTTP is stateless; the graph is not. The checkpointer is what bridges that gap — each `/session/stream` call is one “turn” that resumes a paused state machine.

4. Backend deep dive

4.1 The graph (`server/graph.py`)

- Nodes: `memory_read`, `curriculum`, `theory`, `comprehension`, `problem`, `feedback`, `memory_write`. Linear edges except the conditional after `comprehension`.
- `interrupt_before=["comprehension", "feedback"]` — the two human-input points.
- State is a `TypedDict` (`LessonState`, `total=False`) — see `agents/state.py`. Each node returns a partial dict that `LangGraph` merges into state.
- `thread_config(session_id) = {"configurable": {"thread_id": session_id}}` — this is what isolates one user’s lesson from another’s in the checkpointer.

4.2 Checkpointing & the pooler (the subtle production story)

- Checkpointer: `AsyncPostgresSaver` from `langgraph-checkpoint-postgres`, backed by a **psycopg_pool.AsyncConnectionPool** (constructed in `setup_graph`, torn down in `teardown_graph`, both wired to the FastAPI lifespan).
- **Why a pool, not a single connection:** a single shared `psycopg` connection serializes all checkpoint traffic and can’t recover if the connection drops; a pool gives each operation its own connection and recycles dead ones. (*This was a hardening fix — see §8.*)
- **pgbouncer gotcha:** Supabase’s connection string points at the **transaction pooler (pgbouncer)**, which doesn’t keep server-side prepared statements alive across checkouts. So:
 - `asyncpg` side (`db.py`): `statement_cache_size: 0`.
 - `psycopg/checkpointer` side (`graph.py`): `prepare_threshold: None`.
 - Both disable prepared statements for the same reason. This is a classic “why does it work locally but fail intermittently in prod” story — good to be able to tell it.

4.3 Two persistence layers (don’t conflate them)

Layer	What	Source of truth for	Managed by
Checkpoint state	Full graph state per <code>thread_id</code>	What the live lesson page renders	LangGraph checkpointer
App tables	<code>sessions</code> / <code>messages</code> / <code>user_memory</code> / <code>traces</code>	History, dashboard, durable artifact	SQLAlchemy (<code>models.py</code>)

`messages` is written **once at session end** (`_finalize = messages_from_state`) — it’s the durable, exportable artifact, not the live transcript. `lessons.py:build_steps` reconstructs the page’s sections from checkpoint values.

4.4 Auth (server/deps.py)

- HTTPBearer(auto_error=True) on every protected router; only /health is open.
- _verify_token handles **two** token shapes:
 - Modern Supabase asymmetric JWT (ES256/RS256) → fetch signing key from the project **JWKS** endpoint (PyJWKClient, cached).
 - Legacy HS256 → verify with the shared SUPABASE_JWT_SECRET.
 - audience="authenticated" enforced; exp checked by PyJWT; unknown alg → 401. Fails **closed** (missing config → 401/500, never “allow”).
- Authorization is enforced at the query level: every session/memory lookup is scoped to user.user_id(_load_owned_session, _memory_for_topic). No IDOR.

4.5 Streaming (sse-starlette)

- EventSourceResponse(_event_stream(...)) — an async generator yielding {"data": json} dicts.
- The graph is streamed with stream_mode="custom"; nodes push events through get_stream_writer(). Event types: section_start, token, section_complete, done, error.
- The terminal error path catches everything (except Exception) and emits a terminal error event so the client never hangs.

4.6 Rate limiting (server/ratelimit.py)

- In-process **token bucket** keyed by user_id (20 actions/min, burst to capacity), applied as a dependency (RateLimitedUserDep) to /start, /stream, /hint — the endpoints that cost LLM tokens.
- **Caveat to volunteer:** state is per-process, so it's correct for a single Fly machine; multi-instance would need Redis. Knowing the limitation is the point.

4.7 Config, logging, tracing

- pydantic-settings Settings (config.py), @lru_cache'd; secrets passed explicitly into LLM clients as SecretStr (no env leakage).
- structlog for structured logs.
- Phoenix/Arize via OpenTelemetry; lesson_span(session_id, user_id, topic) wraps a stream and yields a trace_id stored in traces. Tracing is **optional** — skipped if no collector endpoint is configured, so local dev doesn't require it.
- App **boots in degraded mode** if the DB/graph setup fails (lifespan catches) — it logs and serves rather than crash-looping.

5. Frontend deep dive

5.1 Structure (client/src/app, App Router)

- Route groups: (auth) (login/signup) and (app) (dashboard, history, lesson). The (app)/layout.tsx calls getUser() and redirects unauthenticated users — **defense-in-depth** independent of middleware.
- proxy.ts is the Next 16 middleware (note: Next renamed middleware → proxy convention here). It refreshes the Supabase session, guards routes, and now allows /auth/* through for the PKCE callback.

5.2 Auth & data fetching

- Supabase SSR clients: browser(lib/supabase/client.ts) and server(lib/supabase/server.ts).

- API calls forward the Supabase **access token as a Bearer header** to FastAPI — auth is header-based, so it's **not CSRF-exploitable** (no ambient cookie auth to the API).
- Server Components fetch via `lib/api/server.ts`; client via `lib/api/client.ts`.
- `/auth/callback/route.ts` exchanges the PKCE code for a session (email confirmation flow). *(Added in the hardening pass — see §8.)*

5.3 The streaming hook (`lib/hooks/use-lesson.ts`)

- Owns lesson view + phase state machine: `idle` → `streaming` → `awaiting_check` / `awaiting_solution` → `complete` | `error`.
- Parses SSE events into view updates; uses buffers refs to accumulate streamed tokens, splitting theory text from its embedded `**Check:**` question on the fly.
- **Cancellation:** an `AbortController` per run, aborted on unmount and before starting a new stream, so navigating away doesn't leak the connection or `setState` on an unmounted component. *(Hardening fix.)*

5.4 Server state (TanStack Query)

- Single `QueryClient` at the root (`providers.tsx`), `staleTime: 30s`.
- Dashboard is force-dynamic (fresh server fetch each navigation) and seeds the React Query cache so the fresh snapshot wins over a stale cached entry. *(Hardening fix — React Query ignores `initialData` when a cache entry already exists.)*

6. Key design decisions & trade-offs

Decision	Chose	Trade-off / why
Workflow engine	LangGraph	Multi-agent + durable interrupts is exactly its sweet spot; the alternative (hand-rolled state machine + manual persistence) reinvents the checkpointer.
Cross-request state	Postgres	One DB for app + graph state; no Redis/extra infra.
Streaming transport	checkpointer SSE, not WebSocket	Cost: coupled to the pooler quirks (§4.2). Agent runs are unidirectional and one-shot; SSE is plain HTTP, auto-reconnect, trivial in FastAPI. WS framing/ping-pong buys nothing here.
Model routing	Different model per agent	Quality where it matters (Sonnet for prose, 4o for code), cost where it doesn't (mini for planning/memory).
ORM	SQLAlchemy 2.x	The RFC named Drizzle (JS-only); SQLAlchemy is the typed async Python equivalent.
Auth	async Supabase	Auth + DB + checkpointer in one provider; SSR helper for Next.
Persisted artifact	messages written at END only	The page renders from checkpoint state live; the messages table is the durable, exportable record.
Rate limiting	In-process token bucket	Zero deps, fine for single instance; explicitly a stopgap before Redis.

Non-goals (v1): RAG/vector store, model fallbacks (`with_fallbacks`), server-side PDF, mobile, multi-tenant orgs. Be ready to say *why each is deferred* (the lesson loop generates pedagogy rather than retrieving it, etc.).

7. Data model (server/models.py)

```
sessions(id, user_id, topic, status[active|completed], created_at, updated_at)
messages(id, session_id→sessions, role[user|agent|system], content, agent_type, section, created_at)
user_memory(id, user_id, topic, status[covered|struggling|suggested], hint_count, last_seen_at)
    └─ UNIQUE(user_id, topic) ← enables atomic ON CONFLICT hint upsert
traces(id, session_id→sessions, phoenix_trace_id, created_at)
```

- users is owned by Supabase Auth; user_id columns hold the JWT sub UUID (no FK to a table we don't manage).
 - messages and traces cascade-delete with their session.
 - The UNIQUE(user_id, topic) constraint backs the **atomic** hint-count increment (INSERT ... ON CONFLICT DO UPDATE hint_count = hint_count + 1) — no read-then-write race.
-

8. Reliability hardening pass (great “tell me about a bug” material)

A recent review-and-fix pass. Each is a crisp story with a clear root cause:

High severity 1. **Checkpoint** used one shared **psycopg** connection ☐ switched to an AsyncConnectionPool (per-op connection + auto-recovery). 2. **Prepared statements** through **pgbouncer** ☐ set prepare_threshold=None to match the asyncpg statement_cache_size=0 workaround; fixes intermittent prepared statement "..." does not exist. 3. **SSE stream** had no **cancellation** ☐ threaded an AbortController end-to-end; aborts on unmount, releases the reader; no more setState-after-unmount or leaked connections.

Medium severity 4. **Comprehension verdict** misgraded decorated output ("**PASSED**".startswith("PASSED") is false) ☐ case-insensitive regex for the verdict token. 5. **Stale dashboard cache** ☐ seed React Query from the fresh force-dynamic server fetch so it wins over a stale entry. 6. **Trailing SSE event** dropped when not blank-line-terminated ☐ final decode() + flush("\n\n") drains the buffer. 7. **Refreshed auth cookies** dropped on **middleware redirects** ☐ copy supabaseResponse cookies onto the redirect. 8. **Missing PKCE /auth/callback route** ☐ added the exchangeCodeForSession handler so email-confirmation signups complete. 9. **No rate limiting on LLM endpoints** ☐ per-user token bucket.

Talking point: these are mostly **concurrency and lifecycle** bugs (shared connection, pooler statements, stream cancellation, cache freshness) — the kind that pass every local test and only surface under real multi-user traffic.

9. Known limitations / what I'd do next

Be honest about these — interviewers respect it, and several are flagged in architecture.md:

- **RFC-vs-code gaps:** the RFC mentions optimistic-update message appends and react-to-print PDF export; the current code streams via the useLesson hook and react-to-print isn't in package.json yet. The tool_call SSE type is reserved but unused.
- **Rate limiter is per-instance** — move to Redis before scaling out.
- **DB TLS** uses ssl=require (encrypt but no cert verification) — should be verify-full with the Supabase CA.
- **No automated eval gate wired to CI yet** — evals/ exists (run.py, scorers.py) but the PR-gate dataset/threshold is aspirational.
- **N+1 on history/dashboard** — user.py reads graph state per session sequentially; bound the query and gather concurrently.

- **Single Fly machine** with auto start/stop — cold starts on first request after idle.
-

10. Interview Q&A bank

Q: How does the agent graph survive across stateless HTTP requests? A: It doesn't keep anything in memory between requests. LangGraph compiles with a Postgres checkpoint; state is keyed by `thread_id = session_id`. The graph interrupts before the comprehension and feedback nodes, persists, and returns. The next `/session/stream` call does `aget_state + update_state(pending_input=...)` and resumes from the checkpoint.

Q: Why interrupt instead of just making each agent its own endpoint? A: The agents share evolving state (memory summary, plan, attempts, prior answers) and a routing policy (re-explain on failed check, cap attempts). Modeling that as a graph keeps the control flow and state in one place; interrupts give human-in-the-loop pauses for free with durable resume. Splitting into endpoints would scatter the state machine across the HTTP layer.

Q: SSE vs WebSocket — defend it. A: The data flow is server → client only and one-shot per turn. SSE is plain HTTP with auto-reconnect and no framing protocol; FastAPI streams it natively. WebSockets add handshake, ping/pong, and reconnection code for bidirectionality I don't use.

Q: Why different models per agent? A: Cost/quality routing. Planning and memory are low-reasoning and run every session → `gpt-4o-mini`. Generated code must be runnable → `gpt-4o`. Explanation and feedback are the product surface and benefit from Claude's prose/critique → `claude-sonnet-4-6`.

Q: Walk me through a concurrency bug you hit. A: The checkpointer originally opened a single psycopg connection shared by the module-global compiled graph. Under concurrent lessons that serializes all checkpoint I/O and can't recover from a dropped connection. I moved it to an `AsyncConnectionPool` so each operation checks out its own connection and dead ones get recycled — and set `prepare_threshold=None` because Supabase's pgbouncer transaction pooler doesn't keep prepared statements alive across checkouts, which was causing intermittent prepared statement does not exist errors.

Q: How do you prevent one user reading another's session? A: Every session/memory query is scoped to `user.user_id` from the verified JWT. The session loader returns 404 (not 403) if the row's `user_id` doesn't match — no IDOR, and no existence oracle.

Q: How is auth verified server-side? A: FastAPI HTTPBearer + `_verify_token`: asymmetric Supabase JWTs via the project JWKS endpoint, or legacy HS256 via the shared secret, with `audience="authenticated"` enforced and exp checked. It fails closed. The Next.js side guards routes in middleware *and* in the (app) layout via `getUser()`.

Q: What's the source of truth for the lesson page? A: The LangGraph checkpoint state. `build_steps` reconstructs the sections from it. The messages table is written only at session end as the durable, exportable artifact — not the live transcript.

Q: How would you scale this? A: The stateless FastAPI layer scales horizontally behind the pooler; the checkpointer is already in shared Postgres. The two things that are currently single-instance are the in-process rate limiter (→ Redis) and idle cold starts on one Fly machine (→ keep min machines warm / more instances). LLM cost/latency is the real ceiling — that's what the eval pipeline and per-model routing exist to manage.

Q: How do you know the AI output is good? A: Tracing every node to Phoenix gives per-agent latency/cost and full trajectories. The intended quality bar is an eval suite (evals/) scoring theory faithfulness, problem correctness, feedback accuracy, and trajectory, gated in CI on prompt/graph changes — the scaffolding exists; wiring the PR gate is the next step.

Q: Why is the messages table rewritten on every finalize? A: `_finalize` deletes and re-inserts the session's messages from the final checkpoint state so the artifact always reflects the completed lesson

exactly, even if an earlier turn wrote partial rows. It's idempotent-by-replacement.

11. Quick file map (where to point during a screen-share)

Concern	File
Graph assembly, interrupts, checkpoint	server/graph.py
Lesson state shape	server/agents/state.py
Per-agent logic + model choice	server/agents/*.py, server/agents/llms.py
SSE endpoints + finalize	server/routers/session.py
Page reconstruction from state	server/lessons.py
JWT verification	server/deps.py
Rate limiting	server/ratelimit.py
DB engine + pooler workaround	server/db.py
ORM schema	server/models.py
App wiring / lifespan	server/main.py
Streaming client + parser	client/src/lib/api/client.ts
Lesson state machine hook	client/src/lib/hooks/use- lesson.ts
Auth middleware / route guard	client/src/proxy.ts
PKCE callback	client/src/app/auth/callback/route.ts

Pair this with `architecture.md` (the design rationale / build-vs-buy table) for the full story. This doc is the implementation reality + rehearsal material.